SmishGuard: USPS Smishing Incident Analysis

IOC Extraction & Incident Response Documentation

Author: Haytham Abouelfaid
Date: March 3, 2026
Project Type: Cybersecurity / Incident Response
Repository: smishguard-usps-smishing-triage

---

1. Overview

This project documents the analysis of a **smishing (SMS phishing) campaign impersonating the United States Postal Service (USPS)**.

A suspicious SMS message was received that attempted to lure the victim into clicking a malicious link related to a failed package delivery.

The objective of this project was to:

- Analyze the suspicious SMS message
- Extract indicators of compromise (IOCs)
- Document the incident using standard incident response procedures
- Develop a Python tool to automate smishing analysis
- Create detection logic that could be used by security teams

The result of this investigation was the development of **SmishGuard**, a Python-based CLI tool that triages suspicious SMS messages and automatically identifies phishing indicators.

---

2. What is Smishing?

Smishing is a type of phishing attack that uses **SMS text messages** instead of email to trick victims into revealing sensitive information or clicking malicious links.

Attackers commonly impersonate trusted organizations such as:

• Delivery companies (USPS, FedEx, UPS)
• Banks
• Government agencies
• Mobile carriers

These attacks often rely on **urgent language and fake delivery notifications** to create panic and encourage victims to act quickly.

---

3. Incident Summary

A suspicious SMS message was received from an unknown number claiming that a package could not be delivered due to incomplete shipping information.

The message instructed the recipient to click a link to update their delivery details.

The message contained several high-confidence phishing indicators:

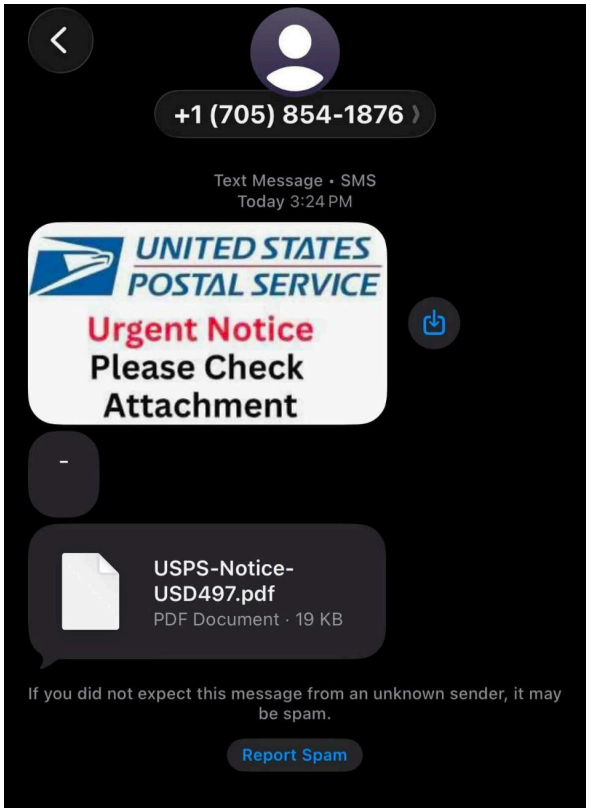
• Random phone number
• Brand impersonation (USPS)
• Urgent delivery message
• Call-to-action link
• Attachment reference

The message followed a pattern consistent with known **delivery-themed smishing campaigns used to steal payment information or credentials.**

---

4. Evidence

Screenshot of the SMS message, message attachment, and sender



---

5. Indicators of Compromise (IOCs)

During analysis, the following indicators were extracted.

Phone Number

+1 (705) 854-1876

URL

USPS.COM/REDELIVERY

Domain

USPS.COM

Attachment

USPS-Notice-USD497.pdf

---

6. Automated Smishing Analysis Tool

To automate the investigation process, a Python command-line tool called **SmishGuard** was developed.

The tool analyzes suspicious SMS messages and extracts:

- Phone numbers
- URLs
- Domains
- Attachment references

It also assigns a **risk score based on known phishing indicators**.

Example command:

```
python -m smishguard.cli --file samples/usps_smish.txt --from "+1 (705) 854-1876" --attachment "USPS-Notice-USD497.pdf"
```

Example output:

```
Risk: HIGH
 Score: 100/100
```

The tool flagged the message because it contained multiple phishing indicators.

---

7. Why the Message Was Flagged

The message triggered several phishing detection rules.

Sender Phone Number

SMS phishing campaigns frequently use random or spoofed phone numbers.

Brand Impersonation

The message impersonates USPS, a common tactic used in delivery scams.

Urgent Language

The message contains urgent delivery language intended to pressure the victim.

Malicious Link

The message includes a redelivery link which may redirect victims to a phishing page.

Attachment Reference

Phishing messages sometimes include attachments that contain malware or malicious instructions.

---

8. Risk Assessment

Severity Level: **High**

Potential risks associated with this attack include:

- Credential harvesting
- Financial fraud
- Malware delivery
- Personal information theft

Because delivery-themed phishing campaigns are extremely common, these messages represent a significant threat to users.

---

9. Safe Attachment Handling Procedure

Suspicious attachments should **never be opened on a host system**.

If analysis is required, the file should be handled in an isolated environment.

Recommended process:

1. Transfer the file to a sandbox environment or virtual machine

2. Compute a SHA256 file hash

Example command (Windows PowerShell):

```
Get-FileHash .\USPS-Notice-USD497.pdf -Algorithm SHA256
```

3. Store the hash in the incident report evidence section

4. Submit the file or hash to a threat intelligence platform such as VirusTotal

This ensures that potential malware does not affect the analyst's system.

---

10. Detection Engineering

A Sigma detection rule was created to identify similar smishing campaigns using common keywords found in delivery scam messages.

Example indicators used in the rule:

- "Urgent Notice"
- "Unable to deliver"
- "USPS"
- "Redelivery"
- "Click here"

This detection rule can be adapted for:

- SIEM platforms
- Email/SMS security gateways
- Threat hunting queries

---

11. Testing and Continuous Integration

The project includes automated testing and linting.

Tools used:

- pytest (unit testing)
- Ruff (linting)
- GitHub Actions (CI pipeline)

This ensures that the tool remains functional and maintains code quality when changes are made.

---

12. Technologies Used

Python
Regex Parsing
Indicator Extraction
Incident Response Documentation
Detection Engineering (Sigma)
GitHub Actions
Pytest
Ruff

---

13. Repository Structure

```
smishguard-usps-smishing-triage
│
├─ smishguard/
│   ├─ cli.py
│   └─ analyzers.py
│
├─ samples/
│   └─ usps_smish.txt
│
├─ docs/
│   ├─ incident_report_template.md
│   ├─ ioc_list.md
│   └─ sigma_smishing_keyword_rule.yml
│
├─ tests/
│
├─ reports/
│   ├─ smishguard-usps-smishing-case-study.pdf
│
├─ requirements.txt
├─ pyproject.toml
└─ README.md
```

14. Conclusion

This project demonstrates how suspicious SMS messages can be analyzed using a structured incident response process.

By combining automated analysis, documentation, and detection rules, SmishGuard provides a lightweight tool for identifying and investigating smishing campaigns.

The project highlights practical cybersecurity skills including:

- Phishing analysis
- Indicator extraction
- Detection engineering
- Incident response documentation
- Security automation using Python